



SPRING-BOOT 3.5 → 4.0 · MINI GUIDE



The Big Jobs: L-Impact Changes

The six changes that eat your sprint

One cut from the full Spring Break Cheat Sheets. Every card verified against Spring Boot 3.5.16 and 4.0.7. Real error output, real fixes, from the spring-break test suite. Steve Poole.

About this mini guide

This is one cut from **Spring Break Cheat Sheets**, the field guide to every breaking change between Spring Boot 3.5 and 4.0. Each card covers one change, verified by running real code on 3.5.16 and 4.0.7. One change, one card, one page: the error you'll see, the fix, and the traps.

HOW TO READ A CARD

TIER	How it fails. Tier 1 won't build, Tier 2 won't start, Tier 3 behaves differently. Tier 3 is the dangerous one: nothing goes red, your data just changes shape.
IMPACT	Effort per occurrence once found. S is find-and-replace, M needs an hour of care, L needs a plan.
OPENREWRITE	The badge means the Boot 4.0 recipe fixes this one for you. Run it before touching anything by hand.
WHAT YOU'LL SEE	The real output, so you can match it against your own build.
THE FIX	Before and after code, plus the steps when a rename isn't enough.
WATCH OUT	The traps that make this change look like a different bug.
VERIFY	The command that proves you're done.

THE SERIES

Spring Batch: 3.5 → 4.0 Eight changes between your jobs and 4.0

Hibernate & JPA: 3.5 → 4.0 Ten data-layer changes, mostly the quiet kind

Security & SSO: 3.5 → 4.0 Seven auth changes you cannot ignore

Data & Messaging: 3.5 → 4.0 Nine changes across your stores and streams

Testing: 3.5 → 4.0 Seven test-suite breaks, one clean build away

Observability & Ops: 3.5 → 4.0 Six changes your dashboards will notice first

Jackson & JSON: 3.5 → 4.0 Nine JSON changes everyone hits

Web & HTTP: 3.5 → 4.0 Eight changes at the HTTP layer

Core Platform & Build: 3.5 → 4.0 Twenty platform changes underneath everything

The Big Jobs: L-Impact Changes The six changes that eat your sprint **THIS GUIDE**

The full guide collects every change in one book, with a complete planning matrix and the OpenRewrite robot list. github.com/spoole167/spring-break



About HeroDevs

HeroDevs keeps critical open source software secure after the community stops patching it. Never-Ending Support (NES) covers CVEs in Spring Boot 3.5, Java, and dozens of other end-of-life frameworks, so your team migrates on its own schedule instead of the vendor's deadline.

This book exists because migrations fail from surprise more often than difficulty. Every card is a fact, not a guess: verified by running real code on Spring Boot 3.5.16 and 4.0.7.

The complete book, every subsystem in one volume, is coming to LeanPub. Watch github.com/spoole167/spring-break for the link.

herodevs.com

The Planning Matrix

Every change in this book, in book order. Impact is the effort per occurrence once found (S small, M moderate, L large), not how many you have. Auto means the OpenRewrite Boot 4.0 recipe handles it.

6 CHANGES

4 × TIER 1 - WON'T BUILD

2 × TIER 2 - WON'T START

TIER	CHANGE	IMPACT	AUTO
1	Jackson 3.0 Group ID Change	L	Y
1	@MockBean / @SpyBean Removed	M	Y
1	OpenSAML 4 Support Removed	L	•
1	Security DSL Rewrite	L	Y
2	OAuth 2.0 Password Grant Completely Removed	L	•
2	Spring Boot Spock Integration Removed	L	•

The Robot List vs The Human List

The left column is what the OpenRewrite Boot 4.0 recipe fixes while you get coffee. The right column needs a person who understands your code. This split is the honest size of the migration: the renames are free, the behaviour changes are the work.

THE ROBOT LIST

Run the recipe, review the diff, done.

Jackson 3.0 Group ID Change

@MockBean / @SpyBean Removed

Security DSL Rewrite

THE HUMAN LIST

Read the card. Check your code. Decide.

OpenSAML 4 Support Removed

OAuth 2.0 Password Grant Completely Removed

Spring Boot Spock Integration Removed

TIER 1

Tier 1 - Won't Build

4 cards

1. Jackson 3.0 Group ID Change
2. @MockBean / @SpyBean Removed
3. OpenSAML 4 Support Removed
4. Security DSL Rewrite

Jackson 3.0 Group ID Change

Tier 1 - Won't Build

Jackson & JSON

Impact: L

🔄 OpenRewrite

WHAT CHANGED

Jackson 3.0 moved from the `com.fasterxml.jackson` Maven group to `tools.jackson`. Spring Boot 4.0's managed dependencies point to Jackson 3.0, so the old group ID no longer resolves. Every import statement using `com.fasterxml.jackson` breaks.

WHY

Jackson's maintainers separated the project identity from the original FasterXML organisation, giving a clean break to drop deprecated APIs and ship Jackson 3.0 as a proper major release.

SCOPE CHECK

Search your codebase for `com.fasterxml.jackson`. Every hit is a coordinate or import that needs updating. If you use `spring-boot-starter-json` or `spring-boot-starter-web`, Jackson arrives transitively. You still need to update explicit dependency declarations and all source-level imports.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/JacksonConfig.java:[3,32]
package com.fasterxml.jackson.databind does not exist
[ERROR] /src/main/java/com/example/JacksonConfig.java:[8,5]
cannot find symbol
symbol: class ObjectMapper
```

THE FIX

pom.xml — explicit Jackson dependency

BEFORE

```
<dependency>
  <groupId>com.fasterxml.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
</dependency>
```

AFTER

```
<dependency>
  <groupId>tools.jackson.core</groupId>
  <artifactId>jackson-databind</artifactId>
</dependency>
```


BEFORE

```
import com.fasterxml.jackson.databind.ObjectMapper;  
import com.fasterxml.jackson.core.JsonProcessingException;
```

AFTER

```
import tools.jackson.databind.ObjectMapper;  
import tools.jackson.core.JsonException;
```

HOW TO FIX

OpenRewrite (recommended).

Run the [Jackson 3 migration recipe](#). Handles group ID changes and import updates across the entire codebase in one pass.

Manual: update Maven coordinates first.

Replace `com.fasterxml.jackson` with `tools.jackson` in all POM or Gradle files, then fix imports file by file.

WATCH OUT

The class renames (JsonSerializer → ValueSerializer, SerializerProvider → SerializationContext) arrive in the same Jackson 3.0 upgrade. Fixing only the group ID leaves broken class references. See [jackson-class-renames](#) for the full list.

Jackson exceptions (`JsonMappingException` , `JsonProcessingException`) are `Serializable` . If you've persisted these anywhere (dead letter queues, error logs), those serialised streams contain the old class names and won't deserialise correctly after migration.

FURTHER INFO

Driven by Jackson 3.0, upstream of Spring Boot 4.0. The FasterXML to tools.jackson rename was announced in 2023 and finalised in Jackson 3.0-rc1. See also: [jackson-class-renames](#), [jackson-exception-hierarchy](#).

spring-break module: [jackson-group-id](#) [Jackson 3 migration guide](#)

@MockBean / @SpyBean Removed

Tier 1 - Won't Build

Testing

Impact: M

🔄 OpenRewrite

WHAT CHANGED

The `@MockBean` and `@SpyBean` annotations from `org.springframework.boot.test.mock.mockito` were deprecated in Spring Boot 3.4 and removed in 4.0. They are replaced by `@MockitoBean` and `@MockitoSpyBean` from `org.springframework.test.context.bean.override.mockito`, which live in Spring Framework 7 itself rather than Spring Boot.

WHY

The new annotations live in Spring Framework's test context, so they work in plain Framework tests as well as Boot tests. Their cleaner bean override mechanism also avoids some of the context caching issues that plagued `@MockBean`.

SCOPE CHECK

Search for `@MockBean` and `@SpyBean` across `src/test`. Every test class using these annotations needs updating. That can be dozens or hundreds of test files. Count them before starting.

WHAT YOU'LL SEE

```
$ mvn clean test
[ERROR] COMPILATION ERROR :
[ERROR] OrderServiceTest.java:[3,48] package
    org.springframework.boot.test.mock.mockito does not exist
[ERROR] OrderServiceTest.java:[18,6] cannot find symbol
    symbol: class MockitoBean
[INFO] BUILD FAILURE
---
The build stops at test-compile. No tests run.
```

THE FIX

Import change

BEFORE

```
import org.springframework.boot.test.mock.mockito.MockBean;
```

AFTER

```
import org.springframework.test.context.bean.override.mockito.MockitoBean;
```

BEFORE

```
import org.springframework.boot.test.mock.mockito.SpyBean;
```

AFTER

```
import org.springframework.test.context.bean.override.mockito.MockitoSpyBean;
```

Annotation usage

BEFORE

```
@MockBean  
private OrderRepository orderRepository;  
  
@SpyBean  
private NotificationService notificationService;
```

AFTER

```
@MockitoBean  
private OrderRepository orderRepository;  
  
@MockitoSpyBean  
private NotificationService notificationService;
```

HOW TO FIX

Find-and-replace annotations.

Replace `@MockBean` with `@MockitoBean` and `@SpyBean` with `@MockitoSpyBean`. Update the imports from `org.springframework.boot.test.mock.mockito` to `org.springframework.test.context.bean.override.mockito`.

OpenRewrite recipe.

The Spring Boot 4.0 OpenRewrite migration recipe handles this rename automatically across the entire test codebase.

Check for `@MockBean` attributes.

If you used `@MockBean(name = "...")` or `@MockBean(classes = ...)`, verify the equivalent attributes exist on `@MockitoBean`. The attribute names may differ slightly.

WATCH OUT

If you upgrade without running `mvn clean`, stale test classes compiled against 3.5 can still load and run. Spring no longer recognises the old annotations, so the mock fields silently stay `null` and the first symptom is a `NullPointerException` mid-test, which looks like a runtime injection failure. Run a clean build and the real compile failure appears.

The new annotations are in the Spring Framework package, not Spring Boot. If you have test utility classes that reference the old annotation by fully qualified name (e.g., in custom annotations or reflection), those references need updating too.

The `@MockitoBean` reset behaviour between tests may differ from `@MockBean`. If you relied on mocks being reset between test methods, verify that behaviour is preserved.

IDE templates and test generators may still produce `@MockBean`. Update your IDE templates after migrating.

OpenSAML 4 Support Removed

Tier 1 - Won't Build

Security & SSO

Impact: L

Spring Security 7 dropped OpenSAML 4 support and requires OpenSAML 5. Code referencing `OpenSaml4AuthenticationProvider` and related classes fails to compile on Boot 4.0.

WHAT CHANGED

Spring Security 7 removed the OpenSAML 4 compatibility classes and requires OpenSAML 5. The SAML support classes (`OpenSaml4AuthenticationProvider`, `OpenSaml4AuthenticationRequestResolver`, etc.) were replaced with OpenSAML 5 equivalents. OpenSAML 5 changed package structures and removed the marshaller/unmarshaller split.

WHY

OpenSAML 4 is no longer maintained. OpenSAML 5 brought significant API improvements, including a simplified object-provider model and better XML security defaults. Maintaining dual support across two major OpenSAML versions was unsustainable.

SCOPE CHECK

Search for `opensaml` in your Maven/Gradle files, and for `OpenSaml4` in Java code. If your application uses SAML 2.0 login (`saml2Login()` in your security config), you are affected. If you only use OAuth 2.0 / OIDC, this card does not apply.

WHAT YOU'LL SEE

```
$ mvn clean compile
[ERROR] COMPILATION ERROR :
[ERROR] SamlConfig.java:[4,63] cannot find symbol
    symbol: class OpenSaml4AuthenticationProvider
[INFO] BUILD FAILURE
---
The build stops at compile. The application never starts.
```

THE FIX

Maven dependency

BEFORE

```
<dependency>
  <groupId>org.opensaml</groupId>
  <artifactId>opensaml-saml-api</artifactId>
  <version>4.3.2</version>
</dependency>
<dependency>
  <groupId>org.opensaml</groupId>
  <artifactId>opensaml-saml-impl</artifactId>
  <version>4.3.2</version>
</dependency>
```

AFTER

```
<dependency>
  <groupId>org.opensaml</groupId>
  <artifactId>opensaml-saml-api</artifactId>
  <version>5.1.2</version>
</dependency>
<dependency>
  <groupId>org.opensaml</groupId>
  <artifactId>opensaml-saml-impl</artifactId>
  <version>5.1.2</version>
</dependency>
```

Java config — authentication provider

BEFORE

```
import org.springframework.security.saml2.provider.service.authentication
    .OpenSaml4AuthenticationProvider;

OpenSaml4AuthenticationProvider provider =
    new OpenSaml4AuthenticationProvider();
```

AFTER

```
import org.springframework.security.saml2.provider.service.authentication
    .OpenSaml5AuthenticationProvider;

OpenSaml5AuthenticationProvider provider =
    new OpenSaml5AuthenticationProvider();
```

Java config — custom response validator

BEFORE

```
OpenSaml4AuthenticationProvider provider =
    new OpenSaml4AuthenticationProvider();
provider.setResponseValidator(responseToken -> {
    Saml2ResponseValidatorResult result =
        OpenSaml4AuthenticationProvider
            .createDefaultResponseValidator()
            .convert(responseToken);
```

AFTER

```
OpenSaml5AuthenticationProvider provider =
    new OpenSaml5AuthenticationProvider();
provider.setResponseValidator(responseToken -> {
    Saml2ResponseValidatorResult result =
        OpenSaml5AuthenticationProvider
            .createDefaultResponseValidator()
            .convert(responseToken);
```

HOW TO FIX

Upgrade OpenSAML dependencies to 5.x.

Update your OpenSAML dependencies to version 5.1.x. If you use Spring Boot's dependency management, the managed version will already be OpenSAML 5 in Spring Boot 4.0. Remove any explicit version overrides pinning to 4.x.

Rename Spring Security SAML classes.

Replace all references to `OpenSam14*` classes with their `OpenSam15*` equivalents. The main classes are `OpenSam15AuthenticationProvider` and `OpenSam15AuthenticationRequestResolver`.

Review custom SAML processing code.

If you directly use OpenSAML APIs (marshallers, unmarshallers, builders), review the OpenSAML 5 migration guide. The marshaller/unmarshaller split was removed and the object provider model changed.

WATCH OUT

OpenSAML 5 requires Java 17+ and has different XML processing requirements. Verify your XML security libraries are compatible.

The Shibboleth Maven repository is still required for OpenSAML 5 artifacts. Make sure your build still has that repository configured.

If you have custom `Sam12AuthenticationRequestContext` processing or assertion decryption, those APIs may have changed signatures in the OpenSAML 5 upgrade.

FURTHER INFO

The change comes from Spring Security 7.0, which Spring Boot 4.0 ships. Affects spring-boot-starter-security consumers using SAML 2.0 login.

[Spring-Break Demo](#) [Spring Security 7 migration](#) [Spring Boot 4.0 Migration Guide](#)

Security DSL Rewrite

Tier 1 - Won't Build

Security & SSO

Impact: L

🔄 OpenRewrite

WHAT CHANGED

Spring Security 7.0 (shipped with Boot 4.0) removed the deprecated `authorizeRequests()` method: use `authorizeHttpRequests()` instead. `antMatchers()`, `mvcMatchers()`, and `regexMatchers()` are replaced by `requestMatchers()`. The `.and()` chaining method is gone; the lambda DSL takes its place.

WHY

The lambda DSL is more readable and avoids a class of configuration bugs where `.and()` chains silently reset the security context. The new `requestMatchers()` API auto-detects whether to use Ant or MVC matching based on your classpath.

SCOPE CHECK

Search for `authorizeRequests()`, `antMatchers()`, `mvcMatchers()`, `.and()` in security config classes, and `.csrf().disable()`. Every `SecurityFilterChain` bean needs reviewing.

WHAT YOU'LL SEE

```
$ mvn compile
[ERROR] /src/main/java/com/example/SecurityConfig.java:[22,14]
  error: cannot find symbol
    symbol:   method authorizeRequests()
    location: variable http of type HttpSecurity
[ERROR] /src/main/java/com/example/SecurityConfig.java:[23,18]
  error: cannot find symbol
    symbol:   method antMatchers(java.lang.String)
[ERROR] /src/main/java/com/example/SecurityConfig.java:[28,10]
  error: cannot find symbol
    symbol:   method and()
```

THE FIX

Security filter chain — before

BEFORE

```
http
    .authorizeRequests()
        .antMatchers("/public/**").permitAll()
        .antMatchers("/admin/**").hasRole("ADMIN")
        .anyRequest().authenticated()
    .and()
    .formLogin()
        .loginPage("/login")
    .and()
    .logout()
        .logoutSuccessUrl("/");
```

AFTER

```
http
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/public/**").permitAll()
        .requestMatchers("/admin/**").hasRole("ADMIN")
        .anyRequest().authenticated()
    )
    .formLogin(form -> form
        .loginPage("/login")
    )
    .logout(logout -> logout
        .logoutSuccessUrl("/")
    );
```

CSRF configuration

BEFORE

```
http.csrf().disable();
```

AFTER

```
http.csrf(csrf -> csrf.disable());
```

HOW TO FIX

Rewrite to lambda DSL.

Replace every `.and()` chain with a lambda-based configuration block. Each security concern (authorization, form login, logout, CSRF) gets its own lambda. See the [Spring Security 7 migration guide](#).

Replace matchers.

Change `antMatchers()` and `mvcMatchers()` to `requestMatchers()`. The new method auto-selects the matching strategy. `regexMatchers()` becomes `requestMatchers(new RegexRequestMatcher(...))`.

VERIFY

App starts and security filter chain initialises without errors

WATCH OUT

The `.and()` removal is the most disruptive part. A long chain with 5+ `.and()` calls means restructuring the whole method. Review the result carefully: it's easy to nest a lambda inside the wrong block.

`requestMatchers()` auto-detects MVC vs Ant matching. If you have Spring MVC on the classpath, it uses MVC matching (which is stricter). If this changes behaviour, use `AntPathRequestMatcher` explicitly.

FURTHER INFO

Driven by Spring Security 7.0. The old DSL was deprecated in Security 5.7/6.0; the lambda DSL has been the recommended replacement since Security 5.2. Affects every spring-boot-starter-security consumer. See also: `websecurity-adaptor-removed`, `auth-default-deny`.

spring-break module: `security-removed-apis` [Spring Security 7 migration guide](#)

TIER 2

Tier 2 - Won't Start

2 cards

1. OAuth 2.0 Password Grant Completely Removed

2. Spring Boot Spock Integration Removed

OAuth 2.0 Password Grant Completely Removed

Tier 2 - Won't Start

Security & SSO

Impact: L

Spring Security 7 removed the OAuth 2.0 Resource Owner Password Credentials grant. Apps that authenticate with it fail at startup or return 401s at runtime.

WHAT CHANGED

The password grant, deprecated in Spring Security 5.x, is gone in Spring Security 7 (shipped with Spring Boot 4.0). The

`ClientRegistration` builder rejects `authorization-grant-type: password`, and the `ResourceOwnerPasswordResourceDetails` class no longer exists.

WHY

The password grant hands user credentials directly to the client application, defeating the point of OAuth delegation. The OAuth 2.1 specification (RFC 9700) formally removed it; Spring Security followed.

SCOPE CHECK

Search for `authorization-grant-type: password` in YAML/properties files and `AuthorizationGrantType.PASSWORD` in Java code. Also search for `ResourceOwnerPasswordResourceDetails` which was the OAuth 2.0 password grant helper class. Any hit means that authentication flow needs redesigning.

WHAT YOU'LL SEE

```
org.springframework.beans.factory.BeanCreationException:
  Error creating bean with name 'securityFilterChain'
...
Caused by: java.lang.IllegalArgumentException:
  The grant type 'password' is not supported.
  Supported grant types are: [authorization_code, client_credentials,
  refresh_token, urn:ietf:params:oauth:grant-type:device_code,
  urn:ietf:params:oauth:grant-type:token-exchange]
  at org.springframework.security.oauth2.client.registration.ClientRegistration$Builder.build(...)
---
APPLICATION FAILED TO START
```

THE FIX

application.yml — client registration

BEFORE

```
spring:
  security:
    oauth2:
      client:
        registration:
          my-api:
            authorization-grant-type: password
            client-id: my-client
            client-secret: secret
        provider:
          my-api:
            token-uri: https://auth.example.com/oauth/token
```

AFTER

```
spring:
  security:
    oauth2:
      client:
        registration:
          my-api:
            authorization-grant-type: authorization_code
            client-id: my-client
            client-secret: secret
            scope: openid,profile
            redirect-uri: "{baseUrl}/login/oauth2/code/{registrationId}"
        provider:
          my-api:
            issuer-uri: https://auth.example.com
```

Java configuration — programmatic registration

BEFORE

```
ClientRegistration.withRegistrationId("legacy")
    .authorizationGrantType(AuthorizationGrantType.PASSWORD)
    .tokenUri("https://auth.example.com/oauth/token")
    .clientId("my-client")
    .clientSecret("secret")
    .build();
```

AFTER

```
ClientRegistration.withRegistrationId("legacy")
    .authorizationGrantType(AuthorizationGrantType.AUTHORIZATION_CODE)
    .authorizationUri("https://auth.example.com/authorize")
    .tokenUri("https://auth.example.com/oauth/token")
    .redirectUri("{baseUrl}/login/oauth2/code/{registrationId}")
    .clientId("my-client")
    .clientSecret("secret")
    .build();
```

HOW TO FIX

Migrate to Authorization Code + PKCE.

Replace the password grant with Authorization Code flow. For browser-based apps, use Authorization Code with PKCE. This is the recommended approach and requires changes to both the client application and possibly the authorization server configuration.

Use Client Credentials for service-to-service.

If the password grant was used for machine-to-machine communication (no actual user), switch to the Client Credentials grant. This does not require user interaction.

Custom token exchange as a last resort.

If you must support legacy systems that send username/password, implement a custom token exchange endpoint on your authorization server that accepts credentials and issues tokens. This keeps the anti-pattern in one controlled place while you plan a proper migration.

VERIFY

Token endpoint returns 200 for auth code flow (not password grant)

WATCH OUT

The grant is gone from the OAuth 2.1 specification itself, so your authorization server (Keycloak, Auth0, Okta) may drop support too. Check both sides.

Mobile apps that used password grant to avoid browser redirects should migrate to Authorization Code with PKCE, which modern mobile OAuth SDKs support natively.

Integration tests that authenticate by posting username/password to the token endpoint will break. Update test helpers to use a test-specific grant flow or mock the security context directly.

FURTHER INFO

Removal tracked in [spring-security#6003](#). Affects `spring-boot-starter-oauth2-client` consumers. See also: `pkce-mandatory`.

[spring-break module: oauth-password-grant-removed](#) [OAuth Password Grant removal issue](#) [Spring Security 7 migration](#)

[Spring Boot 4.0 Migration Guide](#)

Spring Boot Spock Integration Removed

Tier 2 - Won't Start

Spring Boot 4.0 requires Groovy 5; Spock does not support it yet. Spock-based

`@SpringBootTest` specifications stop running.

Testing

Impact: L

WHAT CHANGED

Spring Boot 4.0 upgraded its Groovy baseline to Groovy 5. Spock Framework (as of this writing) requires Groovy 4.x or earlier, so Spring Boot removed the Spock integration.

WHY

The Groovy 5 upgrade aligned Boot with the current Groovy ecosystem. Spock's Groovy 5 support was not ready in time for the Boot 4.0 release, making the integration impossible to ship.

SCOPE CHECK

Find all `.groovy` test files that extend `Specification` and carry `@SpringBootTest`, `@SpringRunner`, or any Spring test slice annotation. These are the tests that break.

WHAT YOU'LL SEE

```
// Spock specification in a Boot 4.0 project:
@SpringBootTest
class MyServiceSpec extends Specification {
    @Autowired MyService service

    def "should do something"() {
        expect: service.doThing() == "result"
    }
}

// Fails at compile or runtime — Spock and Groovy 5 are incompatible.
// Exact error depends on Groovy/Spock versions on the classpath.
```

THE FIX

Spock specification → JUnit 5 migration

BEFORE

```
// MyServiceSpec.groovy
@SpringBootTest
class MyServiceSpec extends Specification {
    @Autowired MyService service

    def "should return result"() {
        expect: service.doThing() == "result"
    }
}
```

AFTER

```
// MyServiceTest.java
@SpringBootTest
class MyServiceTest {
    @Autowired MyService service;

    @Test
    void shouldReturnResult() {
        assertThat(service.doThing()).isEqualTo("result");
    }
}
```

HOW TO FIX

Migrate Spock specifications to JUnit 5.

Rewrite Spock-based Spring integration tests using JUnit 5 and AssertJ. The `given/when/then` blocks map to JUnit 5 methods and AssertJ assertions. Data-driven tests (`where:` blocks) translate to `@ParameterizedTest` with `@MethodSource` or `@CsvSource`.

Wait for Spock's Groovy 5 support if migration is impractical.

If the volume of Spock tests makes immediate migration impractical, track the Spock project's Groovy 5 roadmap and defer the Boot 4.0 upgrade for those modules.

VERIFY

Spock specifications that use `@SpringBootTest` or `@SpringRunner` pass after migrating to JUnit 5 or waiting for Spock's Groovy 5 support

WATCH OUT

Pure unit Spock tests, those that never load a Spring context, still work as long as Groovy compilation itself succeeds. The breakage is in tests that depend on Spring Boot's test infrastructure.

FURTHER INFO

The removal covers the `spock-spring` module and Boot's `@SpringBootTest` support for Spock specifications. Failures appear at Groovy compile time or at runtime before any test method executes.

[Spring Boot 4.0 Migration Guide](#) [Spock Framework GitHub](#)



Not ready to migrate?

Spring Boot 3.5 stops receiving fixes when open-source support ends. You have three sane options. Pick one deliberately.

1. **Pin 3.5 and track CVEs.** Free, but the clock is running and every new CVE is your team's problem to triage.
2. **Migrate incrementally with OpenRewrite.** The Boot 4.0 recipe automates the renames in this book; the behavioural changes still need eyes.
3. **HeroDevs NES for Spring.** Security patches for 3.5 beyond EOL, buying your team time to migrate on your schedule, not the calendar's.

Spring Boot 4.0 Migration Guide • github.com/spring-projects/spring-boot/wiki
OpenRewrite Boot 4.0 recipe • docs.openrewrite.org/recipes/java/spring/boot4
HeroDevs NES for Spring • herodevs.com/support/spring-nes
The test suite behind this book • github.com/spoole167/spring-break